

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN – KHOA HỌC LIÊN NGÀNH
MÔN: **PHƯƠNG PHÁP LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG 2**

HƯỚNG DẪN ĐỒ ÁN
CỜ TƯỚNG

TP.HCM, ngày 14 tháng 09 năm 2026

Contents

1. Mục tiêu của project 3
2. Thiết kế tổng quát 3
3. Xây dựng lớp trừu tượng `QuanCo` 4
4. Xây dựng các lớp `Xe` và `Voi` 4
5. Xây dựng lớp `BanCo` 4
6. Tìm quân tại một vị trí 5
7. Sự phối hợp giữa `QuanCo` và `BanCo` 5
8. Forward declaration là gì? 5
9. Circular dependency là gì? 5
10. Di chuyển con trỏ bằng WASD 6
11. Xử lý phím Enter 6
12. Vẽ bàn cờ 6
13. Tại sao ví dụ hiện tại để tất cả trong một file? 6
14. Khi làm project thật nên chia thành nhiều tập tin 7
15. Mở rộng thêm trò chơi 7
16. Những kiến thức *OOP* có thể học từ *project* 8
17. Kết luận 8

1. Mục tiêu của project

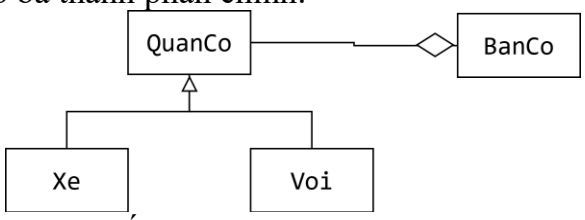
Project này nhằm minh họa cách áp dụng một số khái niệm cơ bản của lập trình hướng đối tượng trong C++, thông qua việc xây dựng một bàn cờ tướng đơn giản chạy trên *console*. Ở phiên bản minh họa ban đầu, chương trình chưa phải là một trò chơi cờ tướng hoàn chỉnh. Ta chỉ xây dựng:

- Một bàn cờ có 10 hàng và 9 cột.
- Một lớp cơ sở trừu tượng `QuanCo`.
- Hai loại quân cụ thể: `Xe` và `Voi`
- Hai quân `Xe` và hai quân `Voi` được đặt trên bàn cờ.
- Người dùng sử dụng W, A, S, D để di chuyển con trỏ.
- Nhấn "Enter" để chọn quân hoặc chọn vị trí muốn di chuyển.
- Phím "Esc" để bỏ chọn và phím "Q" để thoát chương trình.
- Mỗi loại quân tự kiểm tra luật di chuyển của chính nó.

Mục tiêu chính không phải viết một trò chơi hoàn chỉnh mà là quan sát cách các đối tượng phối hợp với nhau.

2. Thiết kế tổng quát

Chương trình hiện tại có ba thành phần chính:



- `QuanCo` là lớp cơ sở, `Xe` và `Voi` kế thừa từ `QuanCo`, chúng là lớp con của `QuanCo`
- `BanCo` quản lý các quân cờ đang tồn tại trên bàn cờ.

Có thể hình dung trách nhiệm như sau:

```
QuanCo
    |
    +-- lưu vị trí
    +-- định nghĩa giao diện chung
    +-- kiemTraNuocDi()
    +-- ve()
```

```
Xe
    |
    +-- luật di chuyển của Xe
```

```
Voi
    |
    +-- luật di chuyển của Voi
```

```
BanCo
    |
    +-- quản lý các quân
    +-- quản lý con trỏ
    +-- chọn quân
    +-- di chuyển quân
    +-- vẽ bàn cờ
    +-- kiểm tra trạng thái các ô
```

Một nguyên tắc quan trọng của thiết kế này là: bàn cờ quản lý trạng thái của ván cờ, còn mỗi quân cờ tự chịu trách nhiệm về luật di chuyển của chính nó.

3. Xây dựng lớp trừu tượng QuanCo

Mọi quân cờ đều có một số đặc điểm chung. Ví dụ:

```
int hang;
int cot;
```

Dù là Xe, Voi, Mã hay Pháo thì chúng đều cần biết mình đang đứng ở đâu. Vì vậy những dữ liệu chung này được đưa vào lớp QuanCo. Ta cũng có các hàm chung như:

```
int getHang() const;
int getCot() const;
void datViTri(int hangMoi, int cotMoi);
```

Tuy nhiên, cách di chuyển của từng quân lại khác nhau. Xe không thể kiểm tra nước đi giống Voi. Do đó QuanCo chỉ yêu cầu rằng:

```
virtual bool kiemTraNuocDi(const BanCo& banCo, int hangMoi, int cotMoi) const = 0;
```

nhưng không đưa ra cách thực hiện cụ thể. Tương tự:

```
virtual string ve() const = 0;
```

Hai hàm được gán *zero* được gọi là *pure virtual function*. Vì vậy QuanCo trở thành một *abstract class* – lớp trừu tượng.

Ta không tạo trực tiếp "QuanCo q;", mà phải tạo một lớp cụ thể kế thừa nó, chẳng hạn: "Xe xe(...); Voi voi(...);". Trong chương trình hiện tại, QuanCo chứa vị trí và khai báo hai phương thức thuần ảo "kiemTraNuocDi()" và "ve()".

4. Xây dựng các lớp Xe và Voi

Hai lớp "class Xe : public QuanCo" và "class Voi : public QuanCo" đều kế thừa QuanCo. Vì QuanCo có các *pure virtual function* nên các lớp con phải cung cấp cài đặt thích hợp. Ví dụ: "bool Xe::kiemTraNuocDi(...)" sẽ kiểm tra:

- Có nằm ngoài bàn cờ không?
- Có đứng yên tại chỗ không?
- Có đi ngang hoặc dọc không?
- Đích đến có quân khác không?
- Trên đường đi có quân cản không?

Trong khi đó "bool Voi::kiemTraNuocDi(...)" kiểm tra:

- Có đi chéo đúng hai ô không?
- Có vượt sông không?
- Mắt Voi có bị chặn không?
- Ô đích có quân khác hay không?

Như vậy "kiemTraNuocDi()" có cùng một ý nghĩa chung nhưng cách thực hiện khác nhau tùy loại quân. Đây là một ví dụ cơ bản về *polymorphism* – đa hình. Trong bản hiện tại, Xe và Voi chỉ khai báo *override* trong *class*, còn phần cài đặt được đặt sau khi BanCo đã được định nghĩa đầy đủ.

5. Xây dựng lớp BanCo

BanCo là lớp chịu trách nhiệm quản lý trạng thái của bàn cờ. Kích thước bàn cờ được xác định bằng:

```
static const int SO_HANG = 10;
```

```
static const int SO_COT = 9;
```

Danh sách các quân được lưu bằng "`vector<unique_ptr<QuanCo>> cacQuan;`". Đây là một điểm rất quan trọng vì ta không viết: "`vector<Xe>`" hay "`vector<Voi>`" mà dùng "`vector<unique_ptr<QuanCo>>`". Nhờ cơ chế đa hình, cùng một *container* có thể quản lý một loại đối tượng (trong lập trình hướng thủ tục gọi là biến) như `Xe` và `Voi` và sau này có thể bổ sung `Ma`, `Phao`, `Tuong`, `Si`, `Tot` mà không cần tạo một danh sách riêng cho từng loại. Trong *constructor* của `BanCo`, chương trình hiện tại tạo hai đối tượng `Xe` và hai đối tượng `Voi` bằng "*`make_unique`*", đồng thời đặt con trỏ ban đầu tại (9, 0).

6. Tìm quân tại một vị trí

Một thao tác mà bàn cờ thường xuyên cần thực hiện là: tại hàng *X*, cột *Y* hiện có quân nào không? Hàm "`QuanCo* timQuan(int hang, int cot) const;`" duyệt danh sách "`cacQuan`" và so sánh tọa độ. Nếu tìm thấy quân thì "`return quan.get();`", nếu không thì "`return nullptr;`".

Từ đó có thể xây dựng hàm đơn giản hơn "`bool coQuanTai(int hang, int cot) const;`" bằng cách kiểm tra "`return timQuan(hang, cot) != nullptr;`".

Hai chức năng này rất hữu ích vì các quân cờ không cần tự quản lý danh sách quân trên bàn. Chúng chỉ hỏi `BanCo` khi cần biết một vị trí đang trống hay bị chiếm.

7. Sự phối hợp giữa QuanCo và BanCo

Đây là một trong những phần quan trọng nhất của *project*. Một quân `Xe` cần biết "trên đường từ vị trí hiện tại đến vị trí mới có quân nào cản không?". Nhưng bản thân `Xe` không giữ danh sách tất cả quân cờ. Danh sách đó thuộc về "`BanCo`". Do đó "`Xe::kiemTraNuocDi()`" nhận vào "`const BanCo& banCo`" và có thể hỏi "`banCo.coQuanTai(...)`" hoặc "`banCo.duongThangTrong(...)`".

8. Forward declaration là gì?

Trong *project* xuất hiện dòng "`class BanCo;`", trước khi `QuanCo` được định nghĩa. Đây được gọi là *forward declaration* – khai báo trước. Có thể hiểu đơn giản là nói với trình biên dịch "Trong chương trình này có một lớp tên `BanCo`. Chi tiết của lớp đó sẽ được cung cấp sau.". Nhờ vậy trình biên dịch chấp nhận khai báo "`const BanCo& banCo`" trong phương thức "`QuanCo::kiemTraNuocDi()`" mặc dù tại thời điểm đó nó chưa đọc phần định nghĩa đầy đủ của BanCo. Nói ngắn gọn "`class BanCo;`" chỉ giúp trình biên dịch biết `BanCo` là một kiểu dữ liệu tồn tại, còn "`class BanCo{...};`" mới cho trình biên dịch biết `BanCo` có dữ liệu gì và `BanCo` có phương thức gì

9. Circular dependency là gì?

Trong *project* này tồn tại một quan hệ hơi đặc biệt. `QuanCo` cần biết đến `BanCo` vì phương thức "`kiemTraNuocDi()`" cần nhận bàn cờ. Nhưng `BanCo` lại quản lý "`vector<unique_ptr<QuanCo>>`" tức là `BanCo` cũng cần biết `QuanCo`. Hai lớp phụ thuộc qua lại như vậy được gọi một cách cơ bản là *circular dependency* – phụ thuộc vòng. Trong ví dụ này ta giải quyết bằng cách khai báo "`class BanCo;`" ở phía trên. Sau đó chỉ khai báo "`bool Xe::kiemTraNuocDi(...);`" trong lớp `Xe`. Ta chưa viết thân phương thức tại đó. Sau khi `BanCo` đã được định nghĩa đầy đủ, ta mới viết:

```
bool Xe::kiemTraNuocDi(...) {
    ...
```

```
}
```

Bởi vì lúc này trình biên dịch đã biết các hàm: `BanCo::trongBanCo()`, `BanCo::coQuanTai()` và `BanCo::duongThangTrong()`. Một cách nhớ đơn giản

- *Forward declaration*: "tôi chỉ cần biết lớp này tồn tại."
- *Full definition*: "bây giờ tôi cần biết lớp này có gì bên trong."

Đây là vấn đề rất thường gặp khi bắt đầu chia chương trình C++ thành nhiều lớp.

10. Di chuyển con trỏ bằng WASD

`BanCo` lưu vị trí con trỏ "`int hangConTro;`" và "`int cotConTro;`". Ví dụ khi nhấn *W* thì "`if (hangConTro > 0) hangConTro--;`". Ta chỉ di chuyển con trỏ chứ chưa di chuyển quân. Đây là điểm cần phân biệt các phím "WASD" là để di chuyển các quân cờ còn phím "enter" là để chọn quân/yêu cầu di chuyển quân. Phần vòng lặp hiện tại xử lý phím "WASD", "enter", "Esc" và "Q" theo mô hình này.

11. Xử lý phím Enter

Phương thức "`void BanCo::xuLyEnter();`" có thể chia *logic* thành ba trường hợp.

- Trường hợp 1 khi con trỏ đang nằm trên một quân: Ta chọn quân đó "`quanDangChon = quanTaiConTro;`". Nếu trước đó đã chọn quân khác thì việc này cũng đồng thời chuyển lựa chọn sang quân mới.
- Trường hợp 2 khi con trỏ nằm trên ô trống nhưng chưa chọn quân thì ta không làm gì.
- Trường hợp 3 khi đã chọn quân và con trỏ nằm trên ô trống. Ta yêu cầu quân đang chọn kiểm tra nước đi bằng "`quanDangChon->kiemTraNuocDi(*this, hangConTro, cotConTro);`". Nếu hợp lệ thì thực hiện "`quanDangChon->datViTri(hangConTro, cotConTro);`". Nếu không hợp lệ thì trạng thái bàn cờ được giữ nguyên.

Đây cũng là nơi thể hiện đa hình rất rõ, nghĩa là `BanCo` chỉ gọi "`quanDangChon->kiemTraNuocDi(...)`", nó không cần viết đại loại như

```
if (quan là Xe) kiểm tra luật Xe;
else if (quan là Voi) kiểm tra luật Voi;
```

C++ sẽ tự gọi cài đặt tương ứng với loại đối tượng thực sự đang được trỏ tới.

12. Vẽ bàn cờ

Trong phiên bản minh họa, bàn cờ được vẽ bằng các ký tự *console* đơn giản. Cụ thể quân `Xe` được biểu diễn (X) và quân `Voi` là (V). Con trỏ và trạng thái chọn quân cũng được biểu diễn bằng các ký tự như {(X)}, [(X)], {+}.

Cách vẽ này không nhằm tạo giao diện đẹp mà để tập trung vào thiết kế chương trình.

Sau này có thể thay phần *console* bằng một thư viện đồ họa mà gần như không cần thay đổi luật di chuyển của các quân.

Phần vẽ hiện tại duyệt từng hàng, từng cột, tìm quân tại ô tương ứng và quyết định cách hiển thị dựa vào vị trí con trỏ và `quanDangChon`.

13. Tại sao ví dụ hiện tại để tất cả trong một file?

Trong *project* minh họa này, toàn bộ mã nguồn đang được đặt trong một tập tin .cpp. Mục đích là giúp người mới học có thể nhìn toàn bộ quan hệ giữa các lớp trên cùng một trang, tránh phải liên tục chuyển qua nhiều tập tin trong lúc đang tìm hiểu:

```
QuanCo
Xe
Voi
```

```
BanCo
main()
```

Việc đặt chung một tập tin vì vậy chỉ nên xem là lựa chọn nhằm đơn giản hóa bài minh họa. Không nên hiểu rằng đây là cách tổ chức tốt cho một *project* C++ thực tế.

14. Khi làm project thật nên chia thành nhiều tập tin

Khi chương trình bắt đầu lớn hơn, nên chia mỗi thành phần thành tập tin riêng. Ví dụ:

```
CoTuong/
├── main.cpp
│
├── QuanCo.h
├── QuanCo.cpp
│
├── Xe.h
├── Xe.cpp
│
├── Voi.h
├── Voi.cpp
│
├── BanCo.h
└── BanCo.cpp
```

Khi bổ sung hàng loạt các tập tin như “Ma.h”, “Ma.cpp”, “Phao.h”, “Phao.cpp”, “Tuong.h”, “Tuong.cpp”, “Si.h”, “Si.cpp”, “Tot.h”, “Tot.cpp”. Lợi ích là:

- Mỗi lớp có phạm vi trách nhiệm rõ ràng hơn.
- Tập tin ngắn hơn và dễ đọc.
- Dễ sửa một loại quân mà không phải tìm trong một tập tin hàng nghìn dòng.
- Dễ làm việc nhóm, dễ tái sử dụng và dễ kiểm thử.
- Dễ mở rộng chương trình sau này.

Ví dụ:

- QuanCo.h: chứa khai báo `QuanCo`
- Xe.h: chứa khai báo lớp `Xe`
- Xe.cpp: chứa cài đặt luật của `Xe`
- BanCo.h: chứa khai báo `BanCo`
- BanCo.cpp: chứa cài đặt của `BanCo`
- main.cpp: chỉ khởi tạo trò chơi và bắt đầu chương trình

Khi tách tập tin, vấn đề *forward declaration* và *circular dependency* cũng trở nên rõ ràng hơn và là một kiến thức C++ rất hữu ích để học tiếp.

15. Mở rộng thêm trò chơi

Sau khi đã hiểu Xe và Voi, hãy hoàn thiện trò chơi theo yêu cầu cơ bản sau:

- Bổ sung đầy đủ các quân cờ cho bàn cờ tướng
- Tạo một ván cờ hoàn chỉnh cho 2 người chơi
- Tạo cơ chế các quân ăn nhau thế nào (có hiệu ứng càng tốt)
- Có các chế độ đánh với máy và chơi đặc biệt: như cờ tiêu chuẩn, cờ chấp…
- Tạo thêm đồng hồ thi đấu tránh đối thủ suy nghĩ quá lâu

- Lưu lịch sử nước đi (rất có ích cho các phân tích sau này nếu có)
- Kiểm tra kết thúc ván chơi và có hỗ trợ tiếng Việt
- Có giao diện trò chơi như "*new game*, *load game*, *settings*…"

16. Những kiến thức *OOP* có thể học từ *project*

Mặc dù chương trình ban đầu rất nhỏ, nó có thể minh họa khá nhiều kiến thức quan trọng.

- *Encapsulation* – đóng gói
- *Inheritance* – kế thừa, ví dụ "`Xe : public QuanCo`" hay "`Voi : public QuanCo`"
- *Abstraction* – trừu tượng hóa: `QuanCo` chỉ định nghĩa những hành vi mà mọi quân cờ phải có "`kiemTraNuocDi()`" và "`ve()`" mà không quy định cụ thể cách thực hiện.
- *Polymorphism* – đa hình: ta có thể viết "`QuanCo* quanDangChon;`" nhưng đối tượng thực tế có thể là: `Xe`, `Voi`, `Ma`, `Phao`, ... và khi gọi "`quanDangChon->kiemTraNuocDi(...)`", C++ sẽ gọi đúng cài đặt tương ứng.
- *Composition*/*object collaboration*: `BanCo` quản lý nhiều đối tượng `QuanCo`, và các quân lại hỏi `BanCo` về trạng thái bàn cờ khi cần kiểm tra nước đi.
- *Forward declaration*: câu lệnh "`class BanCo;`" cho phép khai báo một quan hệ với BanCo trước khi định nghĩa đầy đủ lớp.
- *Circular dependency*: `QuanCo` cần tham khảo `BanCo`, trong khi `BanCo` lại quản lý `QuanCo`. Việc này buộc ta phải suy nghĩ về thứ tự khai báo và cách phân chia *header*/*source*.

17. Kết luận

Project "Cờ Tướng" *console* này không nhằm tạo ngay một trò chơi cờ tướng hoàn chỉnh. Giá trị chính của nó nằm ở việc sử dụng một bài toán quen thuộc để minh họa cách thiết kế chương trình theo hướng đối tượng. Ở phiên bản đầu tiên, chỉ với các lớp cơ bản `QuanCo`, `Xe`, `Voi`, `BanCo` ta đã có thể quan sát được:

abstract class *inheritance* *virtual function* *pure virtual function* *override* *polymorphism* *unique_ptr* *vector* *forward declaration* *circular dependency* *object collaboration* phân chia trách nhiệm giữa các lớp

Việc toàn bộ chương trình hiện đang nằm trong một tập tin .cpp chỉ nhằm đơn giản hóa bài minh họa và giúp người đọc dễ theo dõi luồng chương trình. Khi phát triển thành một *project* thực tế, một trong những việc nên thực hiện sớm nhất là chia mã nguồn thành các tập tin .h và .cpp riêng cho từng thành phần, sau đó tiếp tục mở rộng từng chức năng theo từng bước nhỏ. Quan trọng nhất là không cố viết toàn bộ trò chơi ngay lập tức. Một cách học hiệu quả hơn là:

thiết kế một phần nhỏ

↓
chạy được
↓
hiểu trách nhiệm của từng class
↓
mở rộng thêm một chức năng
↓
refactor lại cấu trúc khi cần

Từ một chương trình chỉ có hai đối tượng `Xe` và hai đối tượng `Voi`, ta có thể dần phát triển thành một hệ thống cờ tướng hoàn chỉnh với nhiều chế độ chơi, đồng hồ thi đấu, lưu ván cờ, *undo*/*redo*, chơi với máy và thậm chí giao diện đồ họa mà vẫn giữ được phần lõi *OOP* ban đầu.